Prepare the data

In the first step the two the two Datasets (Smart_meter contains the readings of one week in 30 Minutes Intervals and survey_with_smart_meter contains characteristics of households) were loaded in R. After that those were joined to "Smart_meter_data" by using "V1=ID" as Key. It was necessary to remove all inverted commas from the records to read them in. For example, through the search and replace function of Notepad++. In addition, we had to change the coding to "UTF-8-BOM, otherwise it read in incorrect characters. In the next step the 15 Features were extracted, in which only 9 had greatuer influence. So it was necessary to fill up with other columns with a manual transfer of the columns to reach the 15 Features.

```
library(dplyr)
smart_meter <- read.csv2("Smart Meter Data_n.csv", stringsAsFactors = FALSE)
survey_with_smart_meter <- read.csv2("Survey Data_n .csv", stringsAsFactors = FALSE, fileEncoding="UTF-8
smart_meter_data <- left_join(smart_meter,survey_with_smart_meter, by=c("V1"="ID"))
#Extracting features for N_devices
set.seed(123)
library(mlbench)
library(caret)
```

```
## Lade nötiges Paket: ggplot2

## Lade nötiges Paket: lattice
```

```
col <- colnames(survey_with_smart_meter)
#select columns, where
smart_meter_data_train <- smart_meter_data[,! names(smart_meter_data) %in% col[-2], drop = F]
rPartMod <- train(N_devices ~ ., data = smart_meter_data_train, method="rpart", na.action=na.exclude)
rpartImp <- varImp(rPartMod, scale = TRUE)
print(rpartImp)
```

```
## rpart variable importance
##
##   only 20 most important variables shown (out of 337)
##
##      Overall
## V139  100.00
## V143   74.70
## V185   73.31
## V42    72.59
## V135   72.57
## V136   25.56
## V87    25.51
## V279   25.19
## V38    23.65
## V306    0.00
## V167    0.00
## V305    0.00
## V32     0.00
## V60     0.00
```

```
## V141    0.00
## V100    0.00
## V259    0.00
## V235    0.00
## V255    0.00
## V95     0.00
```

```
t_names <- c("V139", "V143", "V185", "V42", "V135", "V136", "V87", "V279", "V38", "V306", "V167", "V305
```

Classification and evaluation

In the next step the data was divided into test and training data. 70% of the data was used for training and the remaining 30% were used for testing.

```
#Split into training and test data (70/30)
n = nrow(smart_meter_data_train)
set.seed(4321)
trainIndex = sample(1:n, size = round(0.7*n), replace=FALSE)
train = smart_meter_data_train[trainIndex ,]
test = smart_meter_data_train[-trainIndex ,]
# Classification and evaluation
library(class)
library(e1071)
```

SVM

Then a column for Devices was added to the training and test data set by using a dummy variable. This was saved as a factor. In the next Step the SVM model was developed by using the 15 Features, which were extracted before. The model was used to classify the devides in the given data. The"result_svm" shows the confusion matrix and the indicators for the performance.

```
#svm
#klassifizieren n devices
trainsvm <- train
trainsvm$N_devicesNeu <- ifelse(trainsvm$N_devices == "Low", 0, ifelse(trainsvm$N_devices == "Medium", 
trainsvm$N_devicesNeu <- as.factor(trainsvm$N_devicesNeu)
testsvm <- test
testsvm$N_devicesNeu <- ifelse(testsvm$N_devices == "Low", 0, ifelse(testsvm$N_devices == "Medium", 1, 
testsvm$N_devicesNeu <- as.factor(testsvm$N_devicesNeu)

svm.fit <- svm(trainsvm$N_devicesNeu ~ V139 + V143 + V185 + V42 + V135 + V136 + V87 + V279 + V38 + V306
testLabels_svm <- predict(svm.fit, testsvm)
#Findings svm
result_svm <- caret::confusionMatrix(data=as.factor(testLabels_svm), reference=as.factor(testsvm$N_devi
result_svm
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction  0  1  2
##          0 60 41 16
##          1 21 29 28
##          2 18 35 52
##
## Overall Statistics
##
```

```
##                Accuracy : 0.47
##                  95% CI : (0.4124, 0.5282)
##     No Information Rate : 0.35
##     P-Value [Acc > NIR] : 1.243e-05
##
##                   Kappa : 0.2069
##
##  Mcnemar's Test P-Value : 0.06162
##
## Statistics by Class:
##
##                      Class: 0 Class: 1 Class: 2
## Sensitivity            0.6061  0.27619   0.5417
## Specificity            0.7164  0.74872   0.7402
## Pos Pred Value         0.5128  0.37179   0.4952
## Neg Pred Value         0.7869  0.65766   0.7744
## Prevalence             0.3300  0.35000   0.3200
## Detection Rate         0.2000  0.09667   0.1733
## Detection Prevalence   0.3900  0.26000   0.3500
## Balanced Accuracy      0.6612  0.51245   0.6409
```

KNN

In the KNN model was no real training phase, because the training was only the saving of the data. Therefore it k=10 was used, which means that every Datapoint of the test data set is compared to the 10 nearest Points of the training data set and classificate to the class what occurs most often.

```r
#knn
testLabels_knn<-knn(train=train[,t_names],
                    test=test[,t_names],
                    cl=as.factor(train$N_devices),
                    k = 10)
#Findings knn
result_knn <- caret::confusionMatrix(data=as.factor(testLabels_knn), reference=as.factor(test$N_devices)
result_knn
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction High Low Medium
##     High     44  13     42
##     Low      19  63     35
##     Medium   33  23     28
##
## Overall Statistics
##
##                Accuracy : 0.45
##                  95% CI : (0.3928, 0.5082)
##     No Information Rate : 0.35
##     P-Value [Acc > NIR] : 0.0002208
##
##                   Kappa : 0.1763
##
##  Mcnemar's Test P-Value : 0.1961417
##
```

```
## Statistics by Class:
##
##                      Class: High Class: Low Class: Medium
## Sensitivity               0.4583     0.6364       0.26667
## Specificity               0.7304     0.7313       0.71282
## Pos Pred Value            0.4444     0.5385       0.33333
## Neg Pred Value            0.7413     0.8033       0.64352
## Prevalence                0.3200     0.3300       0.35000
## Detection Rate            0.1467     0.2100       0.09333
## Detection Prevalence      0.3300     0.3900       0.28000
## Balanced Accuracy         0.5944     0.6839       0.48974
```

Random Guessing

```r
#random
testLabels_random <- sample(0:2, length(test$N_devices), replace=TRUE )
table(testLabels_random, testsvm$N_devicesNeu)
```

```
##
## testLabels_random  0  1  2
##                 0 31 44 31
##                 1 39 37 43
##                 2 29 24 22
```

```r
#Findings random
result_random <- caret::confusionMatrix(data=as.factor(testLabels_random), reference=as.factor(testsvm$
result_random
```

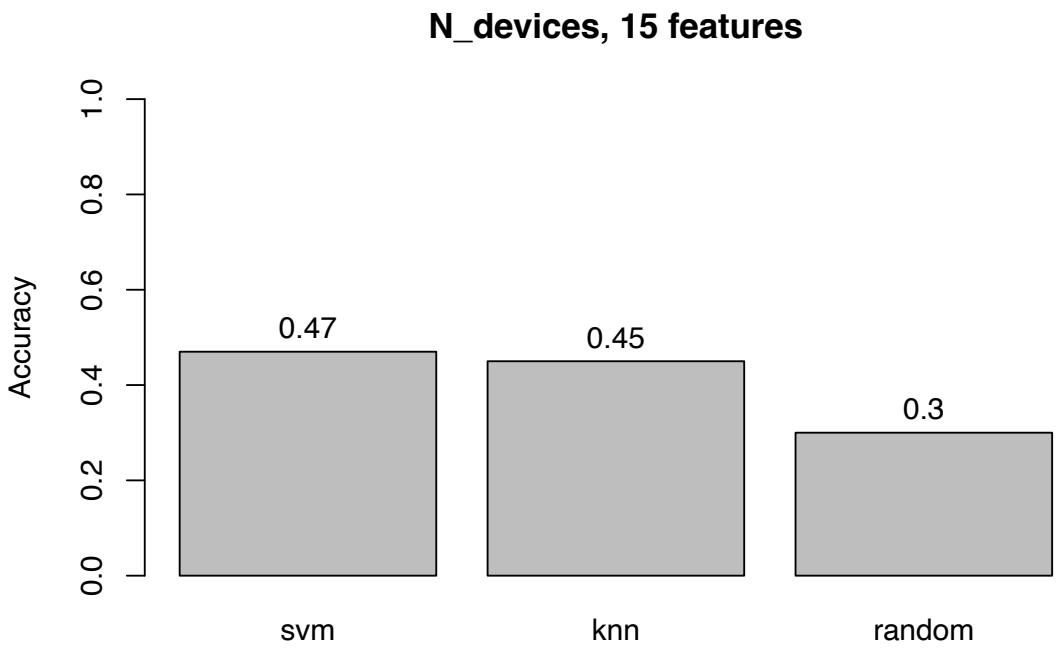
```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction  0  1  2
##          0 31 44 31
##          1 39 37 43
##          2 29 24 22
##
## Overall Statistics
##
##                Accuracy : 0.3
##                  95% CI : (0.2487, 0.3553)
##     No Information Rate : 0.35
##     P-Value [Acc > NIR] : 0.9709
##
##                   Kappa : -0.0533
##
##  Mcnemar's Test P-Value : 0.1241
##
## Statistics by Class:
##
##                      Class: 0 Class: 1 Class: 2
## Sensitivity            0.3131   0.3524  0.22917
## Specificity            0.6269   0.5795  0.74020
## Pos Pred Value         0.2925   0.3109  0.29333
## Neg Pred Value         0.6495   0.6243  0.67111
## Prevalence             0.3300   0.3500  0.32000
```

```
## Detection Rate         0.1033   0.1233  0.07333
## Detection Prevalence   0.3533   0.3967  0.25000
## Balanced Accuracy      0.4700   0.4659  0.48468
```

Result

By comparing the accuracy of the 3 Methods, we see that SVM is the best predictor. Both ML methods are better than random guessing in terms of the accuracy. The SVM model had a specifity of 0.47, the KNN was a little bit below with 0.45 and the random guessing achieved 0.3.

```
#Graphical representation of the results
accuracy_values <-c(result_svm$overall[1], result_knn$overall[1],
                    result_random$overall[1])
classifier_names<-c("svm","knn", "random")
comparison_plot <- barplot(accuracy_values, names.arg=classifier_names, ylim=c(0,1), main= "N_devices, 
text(comparison_plot,accuracy_values+0.05,labels=as.character(round(accuracy_values,3)))
```



Cross Validation

In this section the k-fold cross-validation was used. The data set was randomly distributed in 10 mutually exclusive subsets of similar size. Training and testing is performed 10 times, where in iteration i Di is reserved for testing and training is done with Dataset/Dataset(i)Each sample is used the same number of times for training and once for testing. The confusion matrix wasbe set up with the average results.

```
#Classification/Evaluation(cross fold validation)
smart_meter_data_train <- smart_meter_data[,! names(smart_meter_data) %in% col[-10], drop = F]
rPartMod <- train(N_residents ~ ., data = smart_meter_data_train, method="rpart", na.action=na.exclude)
rpartImp <- varImp(rPartMod, scale = TRUE)
print(rpartImp)
```

```
## rpart variable importance
##
##   only 20 most important variables shown (out of 337)
##
```

```
##      Overall
## V187  100.00
## V186   98.11
## V185   87.02
## V210   73.76
## V114   69.26
## V135   25.34
## V284   23.88
## V271    0.00
## V58     0.00
## V316    0.00
## V228    0.00
## V30     0.00
## V16     0.00
## V254    0.00
## V111    0.00
## V249    0.00
## V220    0.00
## V332    0.00
## V96     0.00
## V63     0.00
```

```
#Manual transfer of the cols, only the first 9 have greater influence, then fill up with other columns
t_names <- c("V187", "V186", "V185", "V210", "V114", "V135", "V284", "V271", "V58", "V316", "V228", "V3

results.svm <- data.frame()
results.knn <- data.frame()
results.random <- data.frame()
smart_meter_data_train$index <- sample(1:10, replace = FALSE)

for(foldIndex in 1:10){
  train.k <- smart_meter_data_train %>% dplyr::filter(index != foldIndex) %>%
    dplyr::select(-index)
  test.k <- smart_meter_data_train %>% dplyr::filter(index == foldIndex) %>%
    dplyr::select(-index)

  train.svm <- train.k
  train.svm$r_new <- NA
  train.svm$r_new <- ifelse(train.svm$N_residents == "Few", 0, 1)
  train.svm$r_new <- as.factor(train.svm$r_new)

  test.svm <- test.k
  test.svm$r_new <- NA
  test.svm$r_new <- ifelse(test.svm$N_residents == "Few", 0, 1)
  test.svm$r_new <- as.factor(test.svm$r_new)

  # svm
  svm.fit <- svm(train.svm$r_new ~ V187+V186+V185+V210+V114+V135+V284+V271+V58+V316+V228+V30+V16+V254+V
  testLabels_svm <- predict(svm.fit, test.svm)

  #save findings svm
  result.svm <- caret::confusionMatrix(data=as.factor(testLabels_svm), reference=as.factor(test.svm$r_n
  results.svm <- rbind(results.svm, data.frame(Acc = result.svm$overall["Accuracy"]))
```

```
  #knn
  testLabels_knn<-knn(train=train.k[,t_names],
                      test=test.k[,t_names],
                      cl=as.factor(train.k$N_residents),
                      k = 10)
  #save findings knn
  result.knn <- caret::confusionMatrix(data=as.factor(testLabels_knn), reference=as.factor(test.k$N_res
  results.knn <- rbind(results.knn, data.frame(Acc = result.knn$overall["Accuracy"]))

  #random
  testLabels_random <- sample(0:1, length(test.svm$r_new), replace=TRUE )
  table(testLabels_random, test.svm$r_new)

  #save findings random
  result.random <- caret::confusionMatrix(data=as.factor(testLabels_random), reference=as.factor(test.s
  results.random <- rbind(results.random, data.frame(Acc = result.random$overall["Accuracy"]))
}
results.svm <- sapply(results.svm[,1], as.numeric)
print(paste("Average Accuracy for svm(N_residents):", mean(results.svm)))
```

```
## [1] "Average Accuracy for svm(N_residents): 0.684"
```

```
results.knn <- sapply(results.knn[,1], as.numeric)
print(paste("Average Accuracy for knn(N_residents):", mean(results.knn)))
```

```
## [1] "Average Accuracy for knn(N_residents): 0.667"
```

```
results.random <- sapply(results.random[,1], as.numeric)
print(paste("Average Accuracy for svm(N_residents):", mean(results.random)))
```

```
## [1] "Average Accuracy for svm(N_residents): 0.503"
```

Significance of the classifiers

Test for statistical significance

First test: H0=No difference of the mean; H1= There is a difference of the means -> P-value is greater than 0.05, so we have to reject H1 -> there is no statistical difference in terms of the mean between SVM and KNN Second test: H0= Difference of the mean of SVM copmared to KNN isn´t greater than 0; H1=Difference of the mean of SVM copmared to KNN is greater than 0 P Value is larger than 0.05, so the mean of SVM isn´t significantly greater than the mean of KNN Third test: H0= Difference of the mean of SVM copmared to random guessing isn´t greater than 0 ; H1=Difference of the mean of SVM copmared to random guessing is greater than 0 -> P Value is smaller than 0.05, so the mean of SVM is significantly greater than the mean of ranndom guessig -> SVM is a better predictor than random guessing Fourth test: H0= Difference of the mean of KNN compared to random guessing isn´t greater than 0; H1=Difference of the mean of KNN copmared to random guessing is greater than 0 P Value is smaller than 0.05, so the mean of KNN is significantly greater than the mean of ranndom guessig -> KNN is a better predictor than random guessing

```
# t-test
t.test(results.svm ,results.knn, paired = TRUE, alternative = "two.sided")
```

```
##
##  Paired t-test
##
## data:  results.svm and results.knn
## t = 0.96022, df = 9, p-value = 0.362
```

```
## alternative hypothesis: true difference in means is not equal to 0
## 95 percent confidence interval:
##  -0.02305005  0.05705005
## sample estimates:
## mean of the differences
##                   0.017
```

```
t.test(results.svm ,results.knn, paired = TRUE, alternative = "greater")
```

```
##
##  Paired t-test
##
## data:  results.svm and results.knn
## t = 0.96022, df = 9, p-value = 0.181
## alternative hypothesis: true difference in means is greater than 0
## 95 percent confidence interval:
##  -0.0154541        Inf
## sample estimates:
## mean of the differences
##                   0.017
```

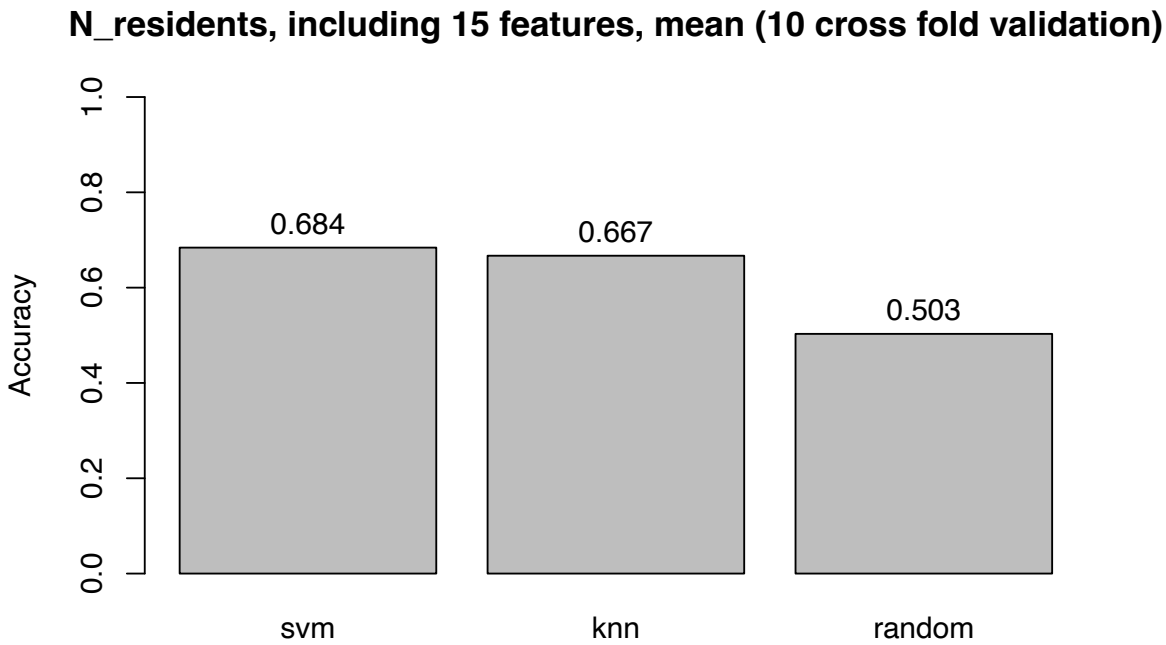
```
t.test(results.svm ,results.random, alternative = "greater")
```

```
##
##  Welch Two Sample t-test
##
## data:  results.svm and results.random
## t = 7.1515, df = 17.751, p-value = 6.312e-07
## alternative hypothesis: true difference in means is greater than 0
## 95 percent confidence interval:
##  0.1370788       Inf
## sample estimates:
## mean of x mean of y
##     0.684     0.503
```

```
t.test(results.knn ,results.random, alternative = "greater")
```

```
##
##  Welch Two Sample t-test
##
## data:  results.knn and results.random
## t = 8.5881, df = 13.838, p-value = 3.236e-07
## alternative hypothesis: true difference in means is greater than 0
## 95 percent confidence interval:
##  0.1303377       Inf
## sample estimates:
## mean of x mean of y
##     0.667     0.503
```

```
#Graphical representation of the results
accuracy.values <-c(mean(results.svm), mean(results.knn), mean(results.random))
classifier.names<-c("svm","knn", "random")
comparison.plot <- barplot(accuracy.values, names.arg=classifier.names, ylim=c(0,1), main = "N_resident
text(comparison.plot,accuracy.values+0.05,labels=as.character(round(accuracy.values,3)))
```



As a result the average Accuracy for the SVM Model was 0.684, for the KNN model 0.667 and for random guessing 0.503. Compared to the holdout method from before the overall accuracy for all models increased by almost 30%, which shows that the advantage of this method over the simple holdout method is that all observations are used both for training and testing. Each observation is used exactly once for testing. The disadvantage is: It takes longer.